

Gmail & Google Workspace Integration

Overview

The Gmail & Google Workspace Integration module provides comprehensive email intelligence capabilities, enabling the AI sales-enablement platform to read, analyze, and draft emails while supporting both human-in-the-loop and fully autonomous email operations. This integration leverages Google's advanced APIs and Gemini AI for sophisticated email processing and generation.

Integration Architecture

Core Gmail Integration Framework

```
class GmailIntegrationFramework:
    def __init__(self):
        self.gmail_client = GmailAPIClient()
        self.workspace_client = GoogleWorkspaceClient()
        self.gemini_client = GeminiAIClient()
        self.email_processor = EmailProcessor()
        self.draft_generator = DraftGenerator()
        self.thread_analyzer = ThreadAnalyzer()
        self.context_manager = EmailContextManager()

    async def initialize(self):
        # Initialize all Google API clients
        await self.gmail_client.initialize()
        await self.workspace_client.initialize()
        await self.gemini_client.initialize()

        # Set up real-time email monitoring
        await self.setup_push_notifications()

        # Start email processing pipeline
        await self.start_processing_pipeline()
```

1. Gmail API Integration

Advanced Gmail Client

Comprehensive Gmail API Client:

```

class GmailAPIClient:
    def __init__(self):
        self.service = None
        self.credentials = None
        self.push_notifications = PushNotificationHandler()
        self.batch_processor = BatchProcessor()

    async def initialize(self):
        # Service account or OAuth 2.0 authentication
        if config.USE_SERVICE_ACCOUNT:
            credentials = service_account.Credentials.from_service_account_file(
                config.GOOGLE_SERVICE_ACCOUNT_FILE,
                scopes=[
                    'https://www.googleapis.com/auth/gmail.readonly',
                    'https://www.googleapis.com/auth/gmail.compose',
                    'https://www.googleapis.com/auth/gmail.modify'
                ]
            )
            # Delegate to user account for domain-wide delegation
            credentials = credentials.with_subject(config.GMAIL_USER_EMAIL)
        else:
            # OAuth 2.0 flow for individual user access
            credentials = await self.oauth_flow()

        self.credentials = credentials
        self.service = build('gmail', 'v1', credentials=credentials)

    async def setup_push_notifications(self):
        # Set up Pub/Sub push notifications for real-time email events
        request = {
            'labelIds': ['INBOX', 'SENT'],
            'topicName': f'projects/{config.GOOGLE_PROJECT_ID}/topics/gmail-notifica-
tions'
        }

        watch_response = self.service.users().watch(
            userId='me',
            body=request
        ).execute()

        return watch_response

```

Real-Time Email Processing:

```

class EmailEventProcessor:
    def __init__(self):
        self.gmail_client = GmailAPIClient()
        self.pubsub_client = pubsub_v1.SubscriberClient()
        self.email_analyzer = EmailAnalyzer()

    async def handle_push_notification(self, message):
        # Decode Pub/Sub message
        message_data = json.loads(message.data.decode('utf-8'))

        if message_data.get('emailAddress') == config.MONITORED_EMAIL:
            history_id = message_data.get('historyId')

            # Get email changes since last history ID
            history = await self.get_history_changes(history_id)

            for history_record in history.get('history', []):
                if 'messagesAdded' in history_record:
                    for message_added in history_record['messagesAdded']:
                        await self.process_new_email(message_added['message'])

    async def process_new_email(self, message_info):
        # Fetch full email content
        message = await self.gmail_client.get_message(message_info['id'])

        # Extract email data
        email_data = await self.extract_email_data(message)

        # Analyze email content
        analysis_result = await self.email_analyzer.analyze(email_data)

        # Determine if response is needed
        if analysis_result.requires_response:
            await self.queue_for_response_generation(email_data, analysis_result)

        # Update CRM with email interaction
        await self.update_crm_with_email(email_data, analysis_result)

```

Email Reading and Analysis

Advanced Email Analyzer:

```

class EmailAnalyzer:
    def __init__(self):
        self.sentiment_analyzer = SentimentAnalyzer()
        self.intent_classifier = IntentClassifier()
        self.entity_extractor = EntityExtractor()
        self.urgency_detector = UrgencyDetector()
        self.gemini_client = GeminiAIClient()

    async def analyze(self, email_data):
        # Multi-dimensional email analysis
        analysis_tasks = [
            self.analyze_sentiment(email_data.content),
            self.classify_intent(email_data.content),
            self.extract_entities(email_data.content),
            self.detect_urgency(email_data),
            self.analyze_with_gemini(email_data)
        ]

        results = await asyncio.gather(*analysis_tasks)

        sentiment_result = results[0]
        intent_result = results[1]
        entities_result = results[2]
        urgency_result = results[3]
        gemini_result = results[4]

        return EmailAnalysisResult(
            email_id=email_data.id,
            sentiment=sentiment_result,
            intent=intent_result,
            entities=entities_result,
            urgency=urgency_result,
            gemini_insights=gemini_result,
            requires_response=self.determine_response_requirement(results),
            suggested_actions=self.generate_suggested_actions(results),
            priority_score=self.calculate_priority_score(results)
        )

    async def analyze_with_gemini(self, email_data):
        # Use Gemini for advanced email understanding
        prompt = f"""
        Analyze this email for sales context:

        From: {email_data.sender}
        Subject: {email_data.subject}
        Content: {email_data.content}

        Provide analysis on:
        1. Customer intent and needs
        2. Buying signals or objections
        3. Next best actions
        4. Relationship status
        5. Deal stage implications
        """

        response = await self.gemini_client.generate_content(
            prompt=prompt,
            model="gemini-pro"
        )

        return GeminiEmailInsights(
            customer_intent=response.customer_intent,

```

```
        buying_signals=response.buying_signals,  
        objections=response.objections,  
        next_actions=response.next_actions,  
        relationship_status=response.relationship_status,  
        deal_implications=response.deal_implications  
    )
```

Thread Analysis and Context

Email Thread Analyzer:

```

class ThreadAnalyzer:
    def __init__(self):
        self.gmail_client = GmailAPIClient()
        self.conversation_tracker = ConversationTracker()

    async def analyze_thread(self, thread_id):
        # Get complete email thread
        thread = await self.gmail_client.get_thread(thread_id)

        # Extract all messages in chronological order
        messages = []
        for message in thread['messages']:
            email_data = await self.extract_email_data(message)
            messages.append(email_data)

        # Analyze conversation flow
        conversation_analysis = await self.analyze_conversation_flow(messages)

        # Track relationship progression
        relationship_progression = await self.track_relationship_progression(messages)

        # Identify key topics and themes
        topics = await self.extract_conversation_topics(messages)

        # Determine conversation status
        status = await self.determine_conversation_status(messages)

        return ThreadAnalysisResult(
            thread_id=thread_id,
            message_count=len(messages),
            conversation_analysis=conversation_analysis,
            relationship_progression=relationship_progression,
            topics=topics,
            status=status,
            last_response_time=messages[-1].timestamp if messages else None,
            requires_follow_up=self.requires_follow_up(messages, status)
        )

    async def analyze_conversation_flow(self, messages):
        # Analyze the flow of conversation
        flow_analysis = {
            'response_times': [],
            'sentiment_progression': [],
            'engagement_level': [],
            'topic_shifts': []
        }

        for i, message in enumerate(messages):
            if i > 0:
                # Calculate response time
                prev_message = messages[i-1]
                response_time = (message.timestamp - prev_message.timestamp).total_seconds()

                flow_analysis['response_times'].append(response_time)

                # Track sentiment changes
                sentiment_change = message.sentiment.score - prev_message.sentiment.score

                flow_analysis['sentiment_progression'].append(sentiment_change)

                # Measure engagement level
                engagement = self.calculate_engagement_level(message, prev_message)

```

```
        flow_analysis['engagement_level'].append(engagement)

    return ConversationFlowAnalysis(**flow_analysis)
```

2. Email Drafting and Generation

AI-Powered Draft Generator

Intelligent Draft Generator:

```

class DraftGenerator:
    def __init__(self):
        self.gemini_client = GeminiAIClient()
        self.template_manager = EmailTemplateManager()
        self.personalization_engine = PersonalizationEngine()
        self.tone_adapter = ToneAdapter()

    async def generate_draft(self, context):
        # Gather comprehensive context
        full_context = await self.gather_context(context)

        # Determine appropriate tone and style
        tone_config = await self.tone_adapter.determine_tone(full_context)

        # Generate draft using Gemini
        draft_content = await self.generate_with_gemini(full_context, tone_config)

        # Apply personalization
        personalized_draft = await self.personalization_engine.personalize(
            draft_content, full_context
        )

        # Validate and refine
        refined_draft = await self.validate_and_refine(personalized_draft,
full_context)

        return EmailDraft(
            subject=refined_draft.subject,
            body=refined_draft.body,
            tone=tone_config,
            confidence_score=refined_draft.confidence,
            suggested_send_time=self.suggest_optimal_send_time(full_context),
            requires_review=refined_draft.confidence < 0.8
        )

    async def generate_with_gemini(self, context, tone_config):
        # Construct comprehensive prompt for Gemini
        prompt = self.build_generation_prompt(context, tone_config)

        response = await self.gemini_client.generate_content(
            prompt=prompt,
            model="gemini-pro",
            generation_config={
                "temperature": 0.7,
                "top_p": 0.8,
                "top_k": 40,
                "max_output_tokens": 1024
            }
        )

        return self.parse_generated_email(response.text)

    def build_generation_prompt(self, context, tone_config):
        return f"""
Generate a professional sales email based on the following context:

RECIPIENT INFORMATION:
- Name: {context.recipient.name}
- Company: {context.recipient.company}
- Role: {context.recipient.role}
- Previous interactions: {context.interaction_history}

```

```

CONVERSATION CONTEXT:
- Thread subject: {context.thread_subject}
- Last message: {context.last_message}
- Conversation stage: {context.conversation_stage}
- Key topics discussed: {context.topics}

BUSINESS CONTEXT:
- Deal stage: {context.deal_stage}
- Deal value: {context.deal_value}
- Key pain points: {context.pain_points}
- Proposed solution: {context.solution}

TONE AND STYLE:
- Tone: {tone_config.tone}
- Formality: {tone_config.formality}
- Urgency: {tone_config.urgency}

REQUIREMENTS:
1. Generate both subject line and email body
2. Keep the email concise and actionable
3. Include a clear call-to-action
4. Maintain professional tone while being personable
5. Reference previous conversation points naturally

Format the response as:
SUBJECT: [subject line]
BODY: [email body]
"""

```

Template Management System

Dynamic Template Manager:

```

class EmailTemplateManager:
    def __init__(self):
        self.template_store = TemplateStore()
        self.template_optimizer = TemplateOptimizer()

    async def get_best_template(self, context):
        # Find templates matching the context
        matching_templates = await self.template_store.find_templates(
            industry=context.recipient.industry,
            deal_stage=context.deal_stage,
            email_type=context.email_type,
            tone=context.desired_tone
        )

        if not matching_templates:
            # Generate new template based on successful patterns
            template = await self.generate_template_from_patterns(context)
            await self.template_store.save_template(template)
            return template

        # Select best performing template
        best_template = await self.template_optimizer.select_best_template(
            matching_templates,
            context
        )

        return best_template

    async def generate_template_from_patterns(self, context):
        # Analyze successful email patterns
        successful_emails = await self.get_successful_emails(context)

        # Extract common patterns
        patterns = await self.extract_patterns(successful_emails)

        # Generate template using Gemini
        template_prompt = f"""
        Create an email template based on these successful patterns:

        Common opening patterns: {patterns.openings}
        Effective body structures: {patterns.body_structures}
        High-converting CTAs: {patterns.ctas}
        Successful closing patterns: {patterns.closings}

        Context: {context.email_type} email for {context.deal_stage} stage
        Industry: {context.recipient.industry}

        Create a template with placeholders for personalization.
        """

        response = await self.gemini_client.generate_content(prompt=template_prompt)

        return EmailTemplate(
            name=f"{context.email_type}_{context.deal_stage}_{context.recipient.in-
            dustry}",
            subject_template=response.subject_template,
            body_template=response.body_template,
            placeholders=response.placeholders,
            context_requirements=context,
            performance_score=0.0 # Will be updated based on usage
        )

```

3. Human-in-the-Loop to Full Autonomy

Progressive Autonomy System

Autonomy Level Manager:

```
class AutonomyLevelManager:
    def __init__(self):
        self.confidence_tracker = ConfidenceTracker()
        self.performance_monitor = PerformanceMonitor()
        self.approval_workflow = ApprovalWorkflow()

    async def determine_autonomy_level(self, context, draft):
        # Calculate confidence score
        confidence_score = await self.calculate_confidence_score(context, draft)

        # Check historical performance
        historical_performance = await self.performance_monitor.get_performance(
            email_type=context.email_type,
            recipient_segment=context.recipient.segment,
            deal_stage=context.deal_stage
        )

        # Determine autonomy level
        if confidence_score >= 0.9 and historical_performance.success_rate >= 0.85:
            return AutonomyLevel.FULL_AUTO
        elif confidence_score >= 0.8 and historical_performance.success_rate >= 0.75:
            return AutonomyLevel.REVIEW_BEFORE_SEND
        elif confidence_score >= 0.6:
            return AutonomyLevel.DRAFT_ONLY
        else:
            return AutonomyLevel.HUMAN_REQUIRED

    async def process_email_with_autonomy(self, context, autonomy_level):
        if autonomy_level == AutonomyLevel.FULL_AUTO:
            # Generate and send automatically
            draft = await self.draft_generator.generate_draft(context)
            await self.send_email(draft, context)
            return EmailProcessingResult(status="sent", draft=draft)

        elif autonomy_level == AutonomyLevel.REVIEW_BEFORE_SEND:
            # Generate draft and queue for quick review
            draft = await self.draft_generator.generate_draft(context)
            await self.approval_workflow.queue_for_review(draft, context, priority="high")
            return EmailProcessingResult(status="pending_review", draft=draft)

        elif autonomy_level == AutonomyLevel.DRAFT_ONLY:
            # Generate draft for human editing
            draft = await self.draft_generator.generate_draft(context)
            await self.approval_workflow.queue_for_editing(draft, context)
            return EmailProcessingResult(status="draft_created", draft=draft)

        else: # HUMAN_REQUIRED
            # Alert human to handle manually
            await self.approval_workflow.alert_human_required(context)
            return EmailProcessingResult(status="human_required", draft=None)
```

Approval Workflow System:

```

class ApprovalWorkflow:
    def __init__(self):
        self.review_queue = ReviewQueue()
        self.notification_service = NotificationService()
        self.user_interface = UserInterface()

    async def queue_for_review(self, draft, context, priority="normal"):
        review_item = ReviewItem(
            id=str(uuid.uuid4()),
            draft=draft,
            context=context,
            priority=priority,
            created_at=datetime.utcnow(),
            status="pending_review"
        )

        await self.review_queue.add_item(review_item)

        # Notify appropriate reviewer
        reviewer = await self.select_reviewer(context)
        await self.notification_service.notify_reviewer(reviewer, review_item)

        return review_item

    async def process_review_decision(self, review_item_id, decision, feedback=None):
        review_item = await self.review_queue.get_item(review_item_id)

        if decision == "approve":
            # Send the email
            await self.send_email(review_item.draft, review_item.context)

            # Record successful approval for learning
            await self.record_approval_success(review_item)

        elif decision == "approve_with_changes":
            # Apply changes and send
            modified_draft = await self.apply_changes(review_item.draft, feedback)
            await self.send_email(modified_draft, review_item.context)

            # Learn from the modifications
            await self.learn_from_modifications(review_item, modified_draft, feedback)

        elif decision == "reject":
            # Don't send, learn from rejection
            await self.learn_from_rejection(review_item, feedback)

        # Update review item status
        review_item.status = f"completed_{decision}"
        review_item.completed_at = datetime.utcnow()
        review_item.feedback = feedback

        await self.review_queue.update_item(review_item)

```

4. Google Workspace Integration

Comprehensive Workspace Integration

Google Workspace Client:

```

class GoogleWorkspaceClient:
    def __init__(self):
        self.calendar_client = GoogleCalendarClient()
        self.drive_client = GoogleDriveClient()
        self.docs_client = GoogleDocsClient()
        self.sheets_client = GoogleSheetsClient()

    async def get_meeting_context(self, email_context):
        # Check for related calendar events
        calendar_events = await self.calendar_client.find_related_events(
            attendee_email=email_context.sender_email,
            time_range=(
                datetime.utcnow() - timedelta(days=30),
                datetime.utcnow() + timedelta(days=30)
            )
        )

        # Get shared documents
        shared_docs = await self.drive_client.find_shared_documents(
            email_context.sender_email
        )

        return WorkspaceContext(
            calendar_events=calendar_events,
            shared_documents=shared_docs,
            recent_collaborations=await self.get_recent_collaborations(
                email_context.sender_email
            )
        )

    async def create_follow_up_tasks(self, email_context, ai_recommendations):
        # Create calendar events for follow-ups
        follow_up_events = []

        for recommendation in ai_recommendations.follow_up_actions:
            if recommendation.type == "schedule_meeting":
                event = await self.calendar_client.create_event(
                    summary=recommendation.title,
                    description=recommendation.description,
                    start_time=recommendation.suggested_time,
                    attendees=[email_context.sender_email],
                    location=recommendation.location
                )
                follow_up_events.append(event)

        return follow_up_events

```

Gemini AI Integration

Advanced Gemini Integration:

```

class GeminiAIClient:
    def __init__(self):
        self.client = genai.GenerativeModel('gemini-pro')
        self.safety_settings = self.configure_safety_settings()

    async def analyze_email_with_context(self, email_data, workspace_context):
        # Comprehensive email analysis with workspace context
        prompt = f"""
        Analyze this email in the context of the broader workspace activity:

        EMAIL:
        From: {email_data.sender}
        Subject: {email_data.subject}
        Content: {email_data.content}

        WORKSPACE CONTEXT:
        Recent meetings: {workspace_context.calendar_events}
        Shared documents: {workspace_context.shared_documents}
        Collaboration history: {workspace_context.recent_collaborations}

        Provide comprehensive analysis including:
        1. Email intent and urgency
        2. Relationship to recent meetings/documents
        3. Business context and implications
        4. Recommended response strategy
        5. Suggested follow-up actions
        6. Risk assessment (if any)
        """

        response = await self.client.generate_content_async(
            prompt,
            safety_settings=self.safety_settings,
            generation_config=genai.types.GenerationConfig(
                temperature=0.3, # Lower temperature for analysis
                top_p=0.8,
                top_k=40
            )
        )

        return self.parse_analysis_response(response.text)

    async def generate_contextual_response(self, analysis_result, user_preferences):
        # Generate response based on analysis and user preferences
        prompt = f"""
        Generate an appropriate email response based on this analysis:

        ANALYSIS:
        Intent: {analysis_result.intent}
        Urgency: {analysis_result.urgency}
        Business context: {analysis_result.business_context}
        Recommended strategy: {analysis_result.response_strategy}

        USER PREFERENCES:
        Communication style: {user_preferences.communication_style}
        Tone preference: {user_preferences.tone_preference}
        Signature: {user_preferences.signature}

        Generate a professional response that:
        1. Addresses the sender's intent appropriately
        2. Maintains the preferred communication style
        3. Includes relevant next steps
        4. Reflects the urgency level appropriately
        """

```

```
"""  
  
response = await self.client.generate_content_async(  
    prompt,  
    generation_config=genai.types.GenerationConfig(  
        temperature=0.7, # Higher temperature for creative response  
        top_p=0.9,  
        top_k=50  
    )  
)  
  
return self.parse_generated_response(response.text)
```

5. Performance Monitoring and Optimization

Email Performance Analytics

Email Performance Tracker:

```

class EmailPerformanceTracker:
    def __init__(self):
        self.metrics_collector = MetricsCollector()
        self.analytics_engine = AnalyticsEngine()

    async def track_email_performance(self, email_id, email_data):
        # Track various performance metrics
        metrics = {
            "sent_time": email_data.sent_time,
            "response_time": None,
            "response_received": False,
            "meeting_scheduled": False,
            "deal_progression": False,
            "sentiment_score": None
        }

        # Monitor for responses
        response_monitor = ResponseMonitor(email_id)
        await response_monitor.start_monitoring()

        # Track business outcomes
        outcome_tracker = OutcomeTracker(email_id, email_data.thread_id)
        await outcome_tracker.start_tracking()

        return PerformanceTrackingSession(
            email_id=email_id,
            metrics=metrics,
            response_monitor=response_monitor,
            outcome_tracker=outcome_tracker
        )

    async def analyze_performance_trends(self, time_period):
        # Analyze email performance trends
        performance_data = await self.metrics_collector.get_performance_data(
            time_period
        )

        trends = await self.analytics_engine.analyze_trends(performance_data)

        return PerformanceTrends(
            response_rate_trend=trends.response_rate,
            meeting_booking_trend=trends.meeting_bookings,
            deal_progression_trend=trends.deal_progression,
            optimal_send_times=trends.optimal_send_times,
            best_performing_templates=trends.best_templates,
            improvement_recommendations=trends.recommendations
        )

```

This comprehensive Gmail and Google Workspace integration provides sophisticated email intelligence capabilities while supporting progressive autonomy and maintaining high performance through continuous monitoring and optimization.